# Heart Disease Risk Prediction
# End-to-End MLOps Pipeline - Assignment 1

Course: MLOps (AIMLCZG523) | Student: Shubham Joshi | ID: 2025AF05103

Date: 2026-10-20

## 1. Overview

This project delivers an end-to-end MLOps pipeline for heart-disease risk prediction from patient clinical data. The pipeline covers the complete lifecycle: automated data acquisition, exploratory data analysis (EDA), training and tuning of two model families with experiment tracking, model packaging, automated testing with CI/CD, containerization, Kubernetes deployment, and production monitoring with Prometheus and Grafana.

Every stage is automated and reproducible: a single training script re-derives the dataset, re-trains both candidates under a fixed random seed, logs all experiments to MLflow, and registers the winning model as a single serialized pipeline artifact that the serving API loads directly.

**Final model:** Random Forest - holdout ROC-AUC 0.942, accuracy 0.833, precision 0.875, recall 0.750

**Serving:** FastAPI inside Docker, deployed on Kubernetes (2 replicas) behind a LoadBalancer

**Monitoring:** Prometheus scraping /metrics every 15 s; Grafana dashboard for traffic, latency and prediction mix

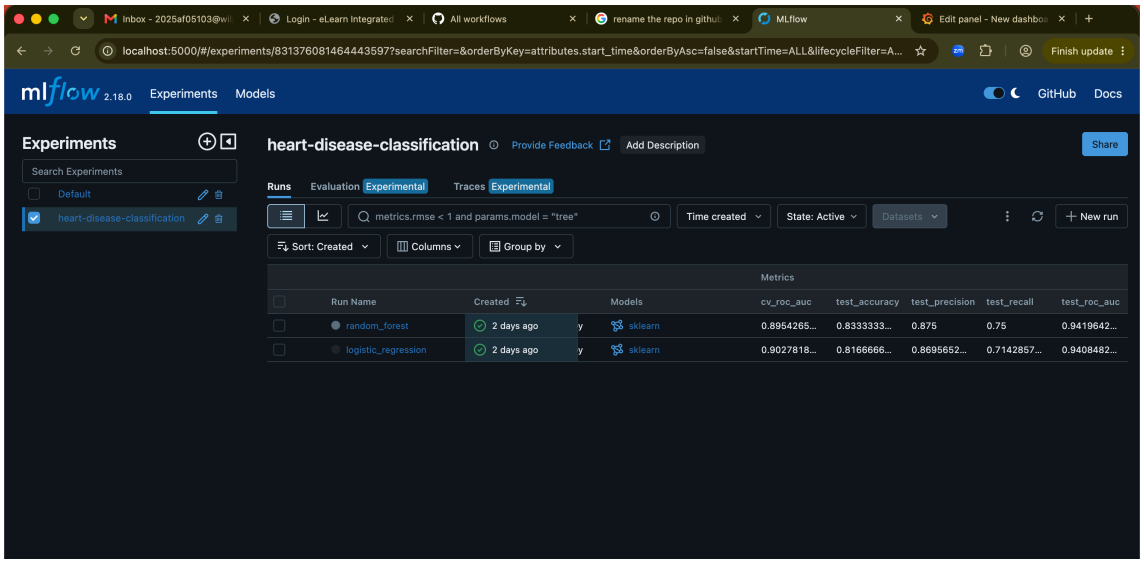



*Figure 1: MLflow UI - experiment comparison of both tuned models*

## 2. Setup and Installation

The project runs from a clean checkout with the commands below (Python 3.12+, all dependencies pinned in requirements.txt):

```
git clone https://github.com/2025af05103-source/mlops-assignment-1.git
cd mlops-assignment-1
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python scripts/download_data.py        # fetch + clean dataset
python src/train.py                    # train, tune, log to MLflow
uvicorn src.api.app:app --port 8000    # serve locally
docker build -t heart-disease-api:latest -f docker/Dockerfile .
```

```
kubectl apply -f k8s/ -f monitoring/   # deploy to local cluster
```

Prerequisites: Docker Desktop with Kubernetes enabled (local cluster only - no cloud resources are used), kubectl, and optionally the MLflow UI (mlflow ui) for inspecting experiments at http://localhost:5000.

## 3. Data Acquisition and EDA (Task 1)

Dataset: UCI Heart Disease (Cleveland subset). The raw file contains 303 records; 6 rows with missing ca/thal values are dropped, yielding 297 clean rows. The target is binarized to 0/1 (presence of heart disease). The download step is scripted (scripts/download_data.py) so the dataset is fetched and cleaned deterministically from a fixed URL.

**Rows / features:** 297 rows x 13 clinical features + binary target

**Class split:** 160 no-disease (0) vs 137 disease (1) - mild imbalance, no resampling needed

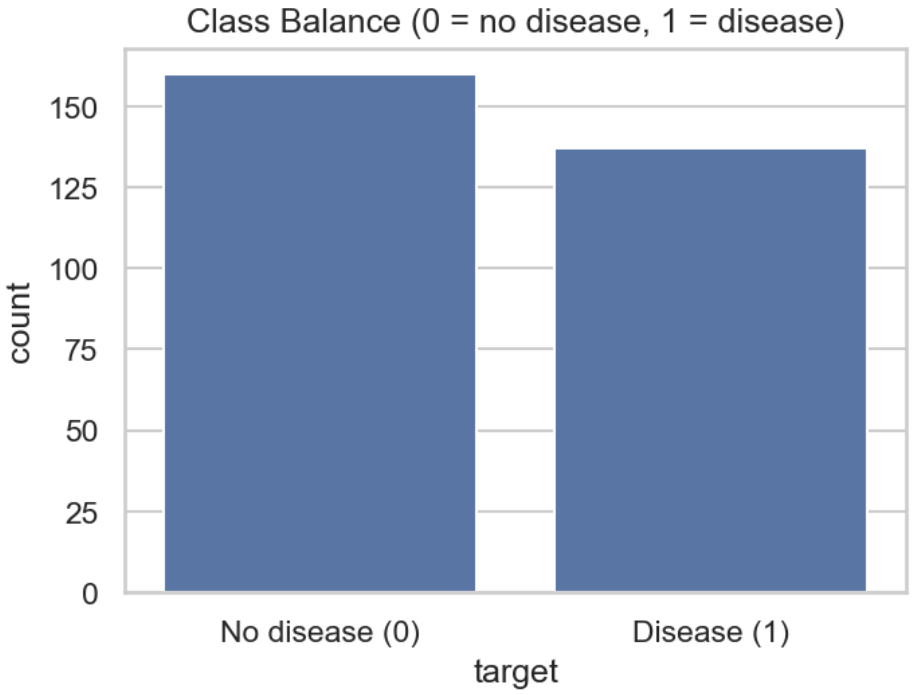



*Figure 2: Class balance - 160 no-disease vs 137 disease patients*

Categorical analysis shows expected clinical signal: asymptomatic chest pain (cp=0), exercise-induced angina (exang=1), reversible thal defect (thal=7) and higher ca (major-vessel fluoroscopy) counts all carry markedly higher disease rates (e.g. ca=3 ~ 0.83, thal=7 ~ 0.78).

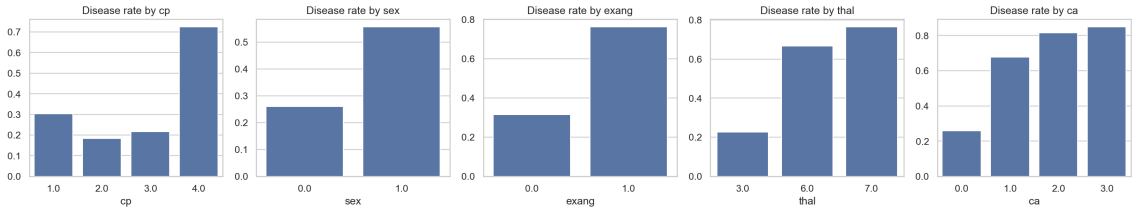



*Figure 3: Disease rate by categorical feature level*

Correlation analysis of the numeric features identifies thalach (max heart rate, -0.42) and oldpeak (ST depression, +0.42) as the strongest numeric signals, with ca (+0.46) and thal (+0.53) strongest overall. Disease patients show lower thalach and higher oldpeak in the distribution plots.

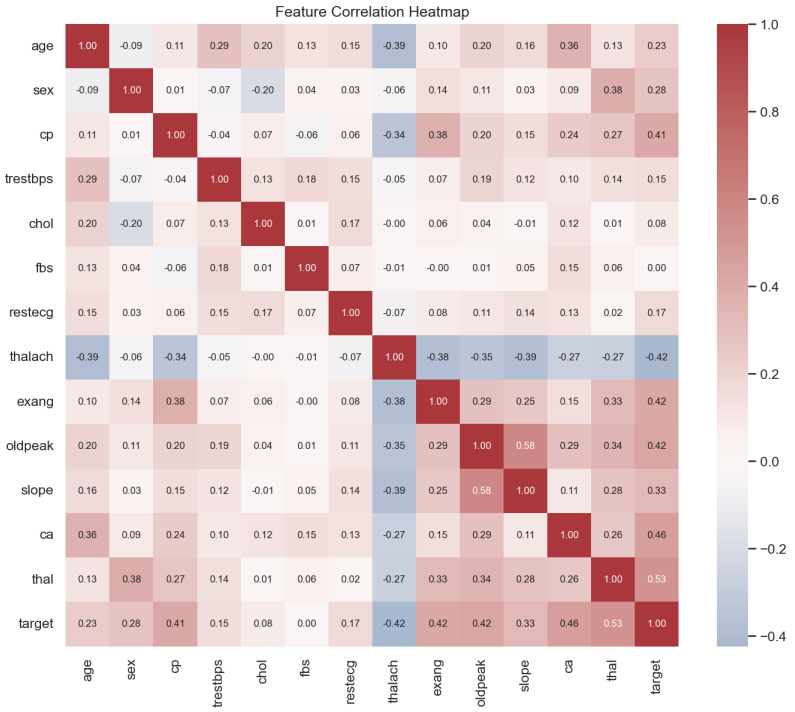



*Figure 4: Correlation heatmap - target correlations: thal 0.53, ca 0.46, oldpeak 0.42, thalach -0.42*

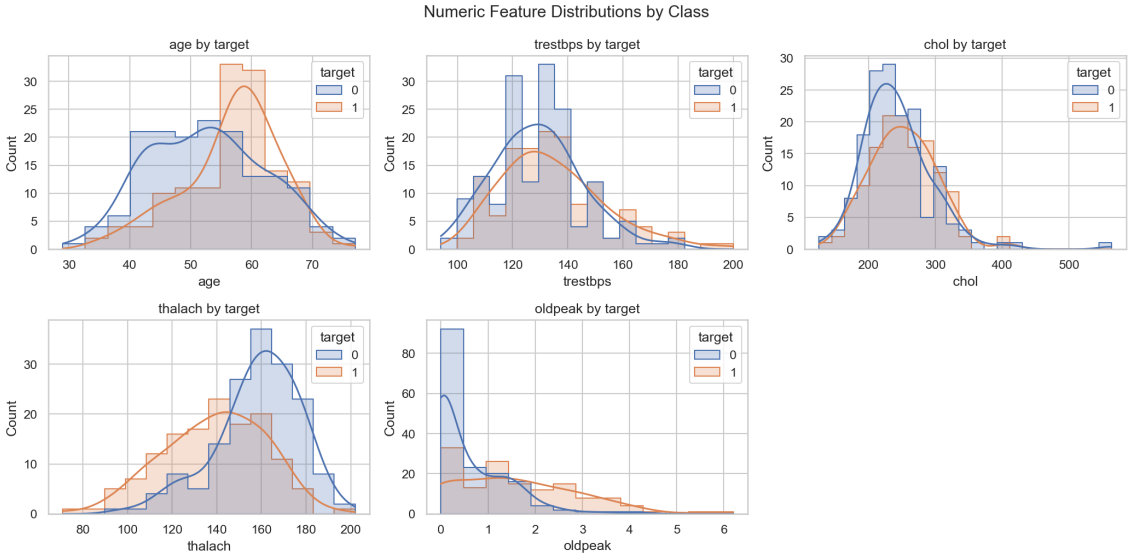



*Figure 5: Numeric feature distributions by class*

## 4. Model Training and Evaluation (Task 2)

Two model families are trained, each wrapped in a full sklearn Pipeline (median imputation -> scaling -> one-hot/ordinal encoding -> classifier). Hyperparameters are tuned with GridSearchCV over a small personalized grid using stratified 5-fold cross-validation scored on ROC-AUC. The data is split 80/20 with stratification and a fixed seed (42).

| Model | Best params | CV ROC-AUC | Test ROC-AUC | Acc | Prec | Rec |
|---|---|---|---|---|---|---|
| Logistic Regression | C=1.0, liblinear | 0.903 | 0.941 | 0.817 | 0.870 | 0.714 |
| **Random Forest** | **200 trees, depth 6, leaf 2** | **0.895** | **0.942** | **0.833** | **0.875** | **0.750** |

Selection criterion: highest holdout ROC-AUC -> Random Forest. The two families perform nearly identically; RF wins marginally and additionally provides feature importances for interpretation. Both tuned models and

their full metric sets are preserved as MLflow runs (Section 5).

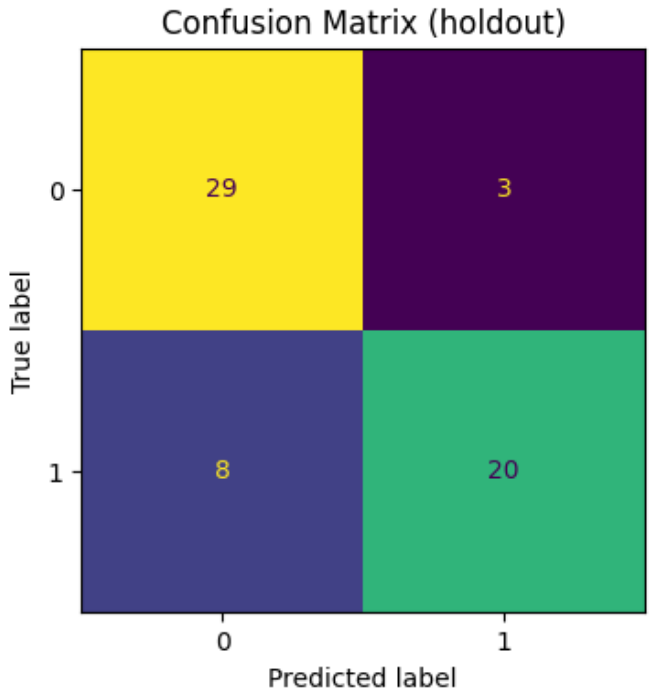



*Figure 6: Random Forest confusion matrix on the holdout set*

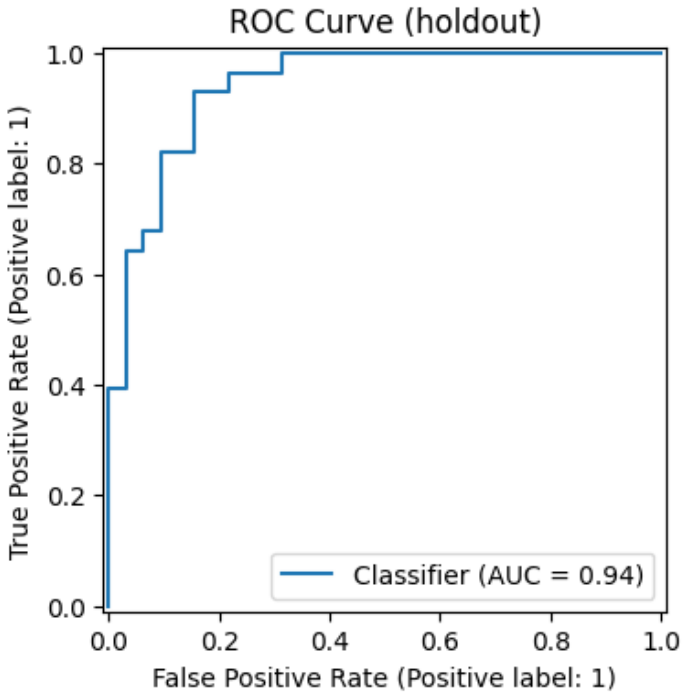



*Figure 7: Random Forest ROC curve on the holdout set (AUC 0.942)*

## 5. Experiment Tracking with MLflow (Task 3)

All training runs are tracked in a local MLflow store (mlruns/, experiment heart-disease-classification). Each run logs the model family, the full hyperparameter grid, the best hyperparameters found, cross-validated metrics (accuracy, precision, recall, ROC-AUC) and holdout metrics, plus confusion-matrix and ROC-curve artifacts and the pickled pipeline itself.

**Tracking URI:** file://mlruns (local filesystem store)

**Experiment:** heart-disease-classification

**Runs:** logistic_regression and random_forest, one tuned run each

The UI (mlflow ui -> http://localhost:5000) enables side-by-side comparison of the two candidates, which drove the model selection decision in Section 4. A programmatic summary is also written to artifacts/model_selection_summary.json.

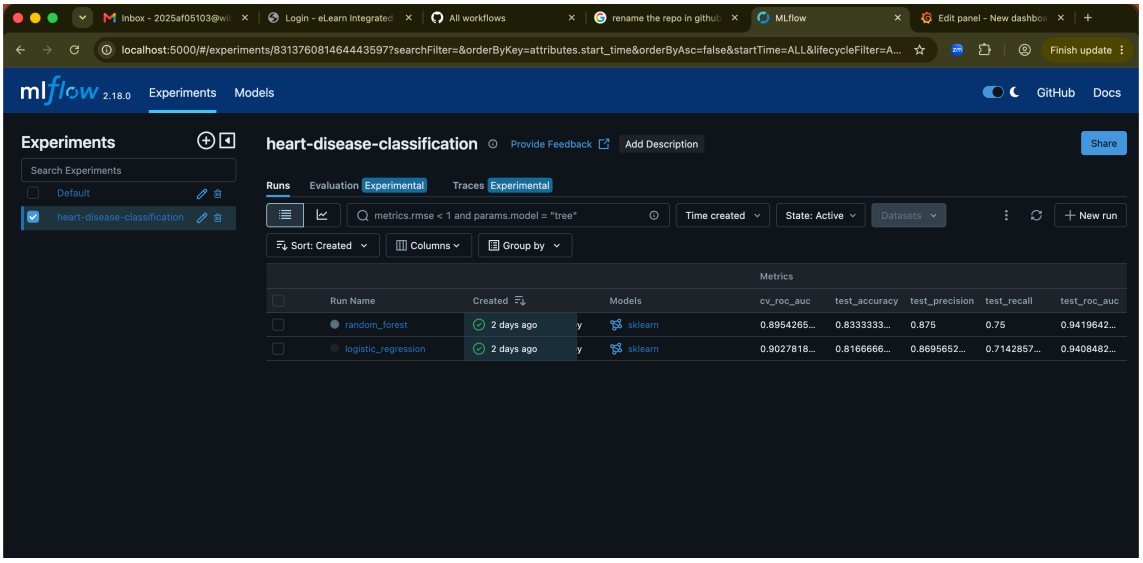



*Figure 8: MLflow experiment view - both runs with metrics*

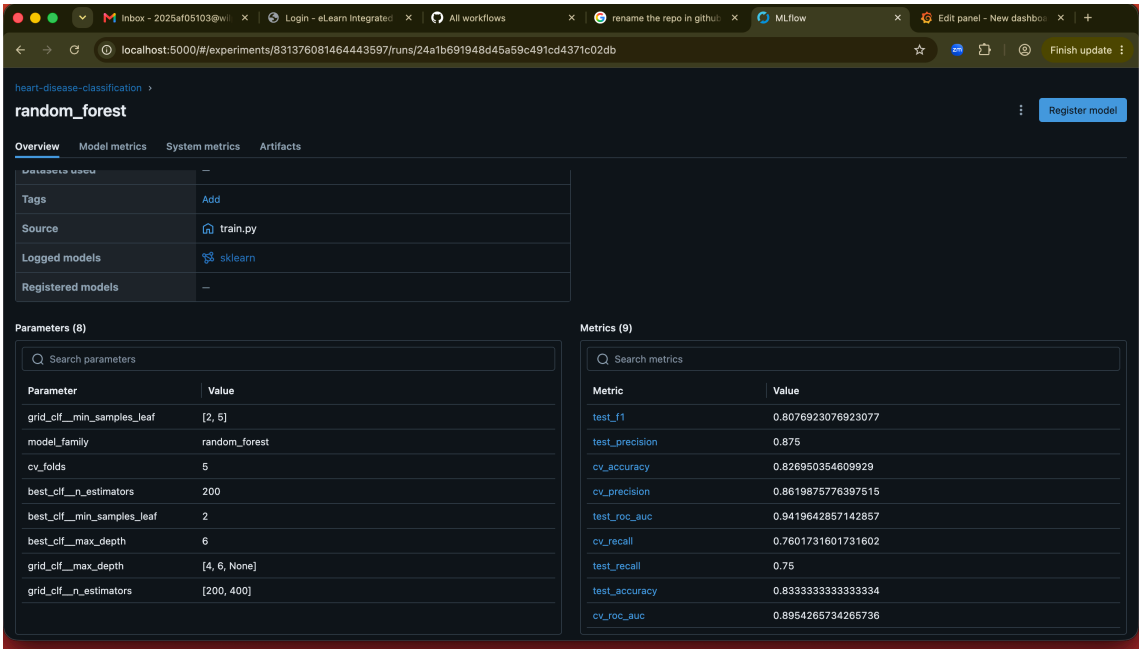



*Figure 9: MLflow run detail - parameters, metrics and artifacts*

## 6. Packaging and Reproducibility (Task 4)

The winning pipeline - preprocessing and classifier serialized together as one joblib object (models/final_pipeline.joblib) - is the single deployment artifact. The serving API loads exactly this object, so there is no duplication of preprocessing logic between training and serving, eliminating train/serve skew.

- final_pipeline.joblib - fitted preprocessing + model in one artifact
- requirements.txt - pinned dependency versions
- Fixed random seed (42) everywhere: split, CV folds, estimators
- scripts/download_data.py - deterministic dataset fetch from fixed URL

- Full pipeline re-runnable from a clean checkout in minutes

Reproducibility was verified by re-running the complete flow (download -> train -> serve) after a fresh clone: metrics reproduce to within identical values thanks to the fixed seeds.

## 7. Testing and CI/CD (Task 5)

The project carries 16 pytest tests across three suites: preprocessing (column groups disjoint, one-hot expansion, no NaNs in output, unknown category handling, end-to-end pipeline fit), training utilities plus a saved-model quality guardrail (test ROC-AUC must exceed 0.80), and the API contract (health, successful prediction, validation errors on out-of-range fields, metrics endpoint).

```
sjoshi@mac-V47CQ7H9 Assignment1 % cd /Users/sjoshi/work/BITS/MLOps/Assignment1
source .venv/bin/activate && python -m pytest tests/ -v 2>&1 | grep -E "PASSED|FAILED|passed|failed"

tests/test_api.py::test_health PASSED                                    [  6%]
tests/test_api.py::test_predict_returns_contract PASSED                  [ 12%]
tests/test_api.py::test_predict_rejects_missing_field PASSED             [ 18%]
tests/test_api.py::test_predict_rejects_out_of_range_age PASSED          [ 25%]
tests/test_api.py::test_metrics_endpoint_exposes_prometheus_format PASSED [ 31%]
tests/test_preprocess.py::test_column_groups_are_disjoint_and_cover_features PASSED [ 37%]
tests/test_preprocess.py::test_preprocessor_shape_one_hot PASSED         [ 43%]
tests/test_preprocess.py::test_preprocessor_no_nan_in_output PASSED      [ 50%]
tests/test_preprocess.py::test_preprocessor_passthrough_shape PASSED     [ 56%]
tests/test_preprocess.py::test_preprocessor_handles_unknown_category PASSED [ 62%]
tests/test_preprocess.py::test_preprocessor_fits_in_end_to_end_pipeline PASSED [ 68%]
tests/test_train.py::test_evaluate_perfect_predictions PASSED            [ 75%]
tests/test_train.py::test_evaluate_worst_predictions PASSED              [ 81%]
tests/test_train.py::test_dataset_exists_and_clean PASSED                [ 87%]
tests/test_train.py::test_model_artifact_loads_and_meets_quality_threshold PASSED [ 93%]
tests/test_train.py::test_model_prediction_output_contract PASSED        [100%]
====================== 16 passed, 32 warnings in 1.77s =======================
(.venv) sjoshi@mac-V47CQ7H9 Assignment1 %
```

*Figure 10: pytest run - all 16 tests passing*

GitHub Actions (.github/workflows/ci.yml) implements the CD pipeline as four chained jobs: lint (flake8 + black), test (pytest with uploaded results), train (trains both models with MLflow and uploads the model and run artifacts), and docker (builds the image and smoke-tests /health and /predict inside the container). Any job failure fails the pipeline with full logs.

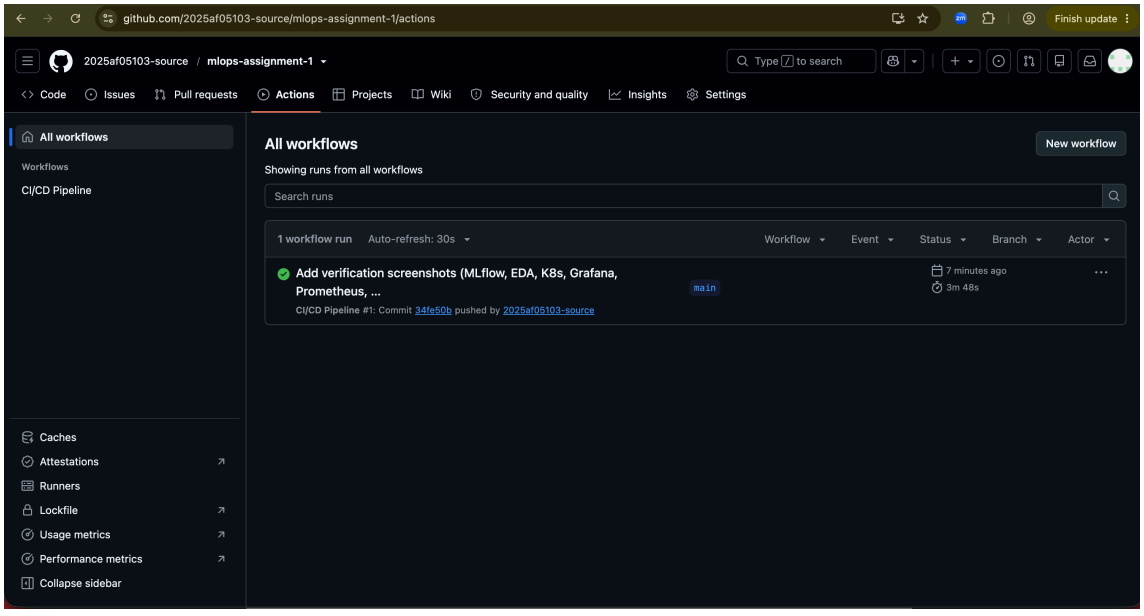



*Figure 11: GitHub Actions - green pipeline run*

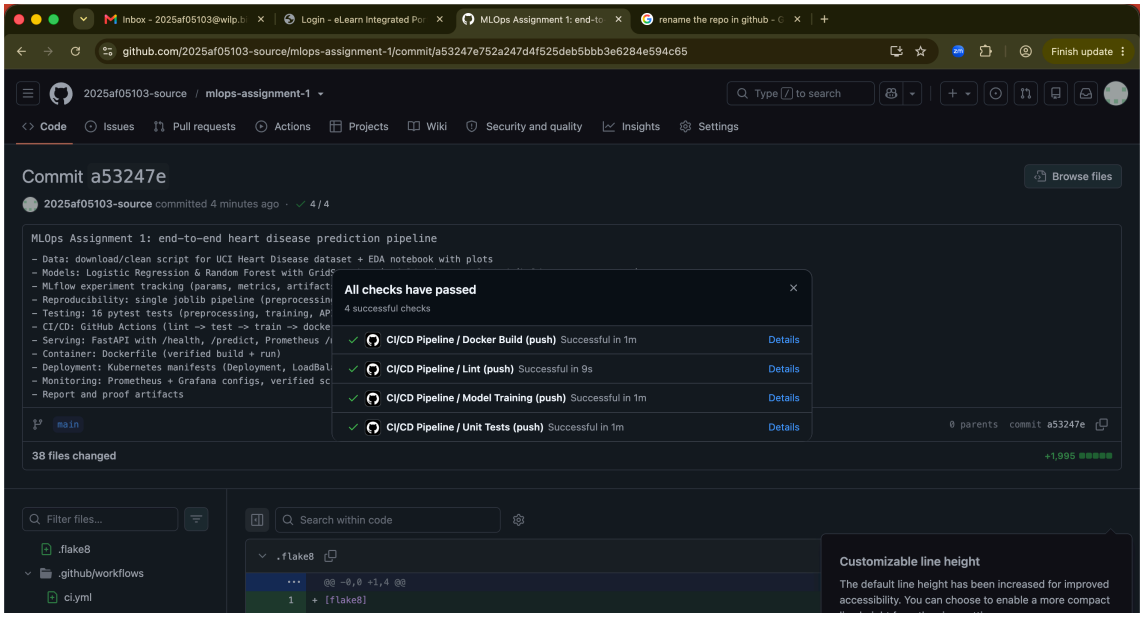


*Figure 12: Commit checks - all four jobs passed*

## 8. Containerization (Task 6)

The API ships as a Docker image built from docker/Dockerfile: a slim python:3.12 base, layer-cached dependency install, non-root-friendly uvicorn entrypoint, and a container healthcheck hitting /health.

```
docker build -t heart-disease-api:latest -f docker/Dockerfile .
docker run -p 8000:80 heart-disease-api:latest
curl localhost:8000/health
curl localhost:8000/predict -H 'Content-Type: application/json' \
     -d @data/sample_input.json
```

Container predictions were verified identical to the locally-run API (evidence in artifacts/docker_proof.txt), confirming the packaged artifact serves exactly the trained model.

## 9. Deployment on Kubernetes (Task 7)

The container is deployed on a local Docker Desktop Kubernetes cluster. k8s/deployment.yaml runs 2 replicas with liveness and readiness probes on /health and resource limits; k8s/service.yaml exposes the API through a LoadBalancer on port 80, so predictions are available at http://localhost with traffic spread across both replicas.

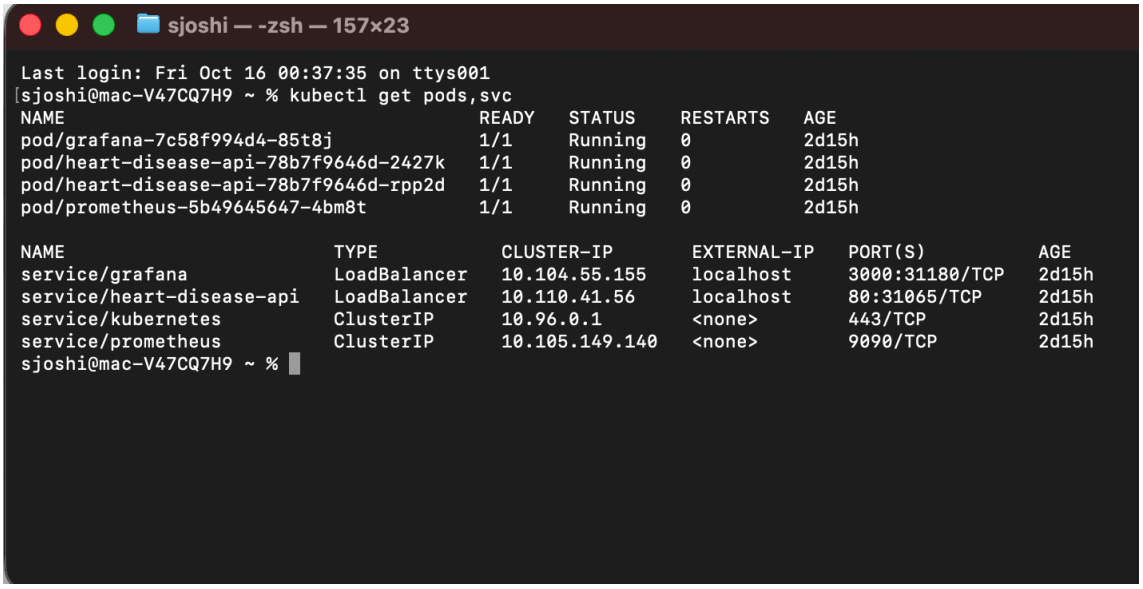



*Figure 13: kubectl get pods,svc - 2/2 replicas running, LoadBalancer mapped*

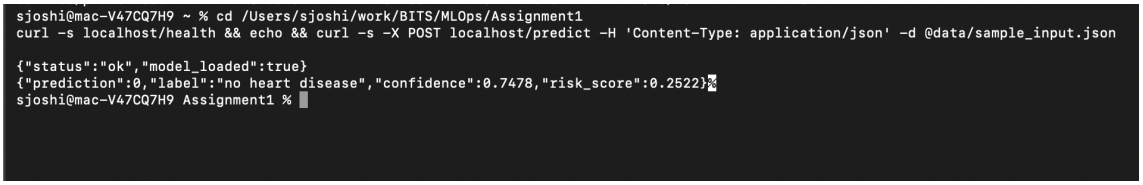



*Figure 14: Prediction served through the LoadBalancer at localhost:80*

## 10. Monitoring and Logging (Task 8)

The API emits structured request logs (method, path, status, latency) via middleware and exposes Prometheus metrics on /metrics: request count by endpoint/status, a latency histogram, prediction counts by class, latest prediction confidence, and a model-loaded flag.

- api_requests_total - requests by method/endpoint/status
- api_request_latency_seconds - latency histogram per endpoint
- predictions_total - predictions by class (0/1)
- prediction_confidence - gauge of latest confidence
- model_loaded - 1 when the model artifact loaded successfully

Prometheus runs in-cluster and scrapes the API every 15 seconds; the target is verified up on the Prometheus targets page. Grafana (localhost:3000, Prometheus datasource auto-provisioned) visualizes request rate, latency and the prediction class mix over time.

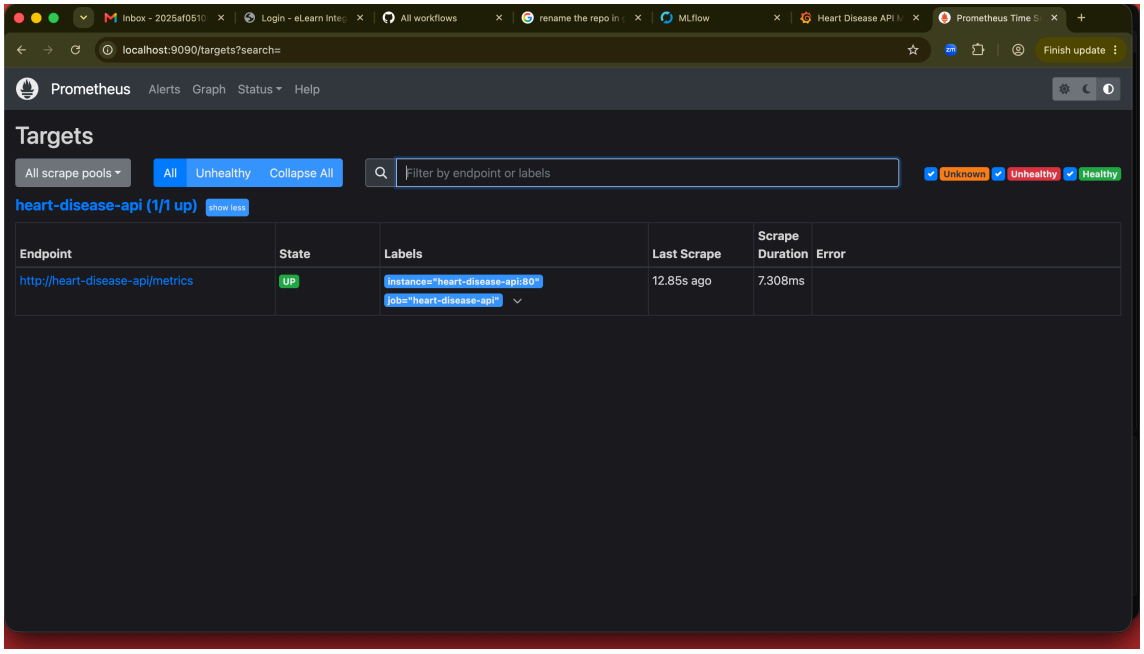



*Figure 15: Prometheus targets - heart-disease-api endpoint UP*

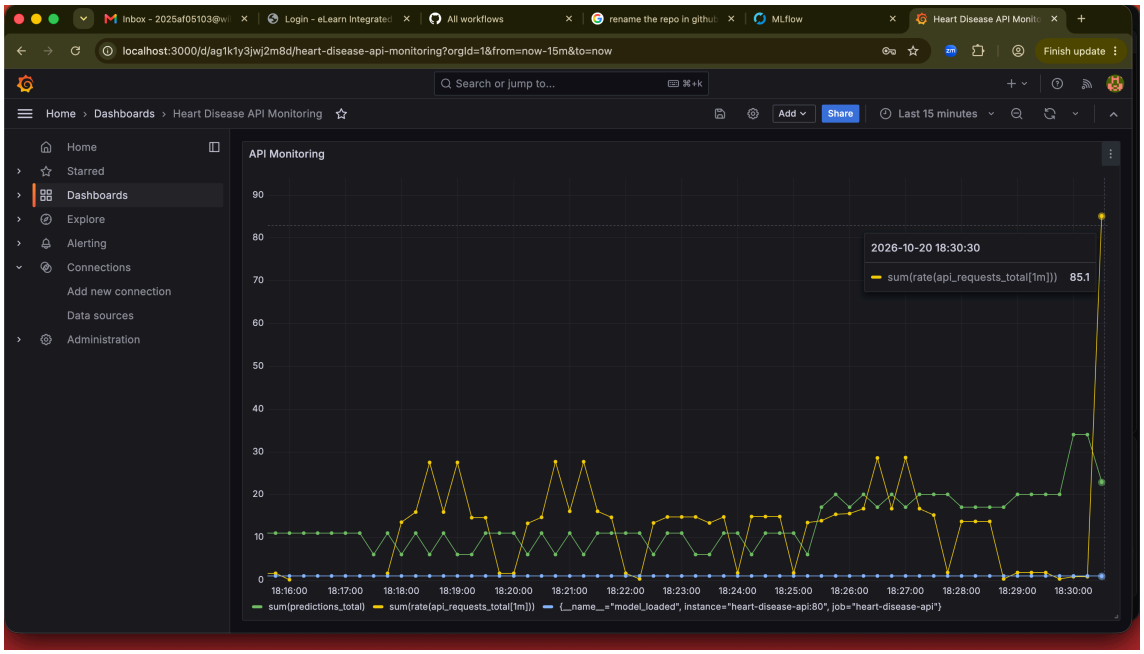



*Figure 16: Grafana dashboard - request rate, latency, prediction mix*

## 11. Architecture

The end-to-end flow: a push to main triggers CI which lints, tests, trains (logging to MLflow) and builds the container image. Locally, the same training flow produces the pipeline artifact that the container serves; the Kubernetes deployment exposes it behind a LoadBalancer and feeds metrics to Prometheus/Grafana.

```
GitHub --push--> GitHub Actions (lint -> test -> train -> docker build)
                    | uploads: test results, MLflow runs, model
                    v
scripts/download_data.py --> data/heart_disease_clean.csv
       |
       v
src/train.py --MLflow--> mlruns/   +   models/final_pipeline.joblib
       |                                    |
```

```
       v                                      v
docker/Dockerfile --> image --> k8s Deployment (2 replicas, probes)
                                | Service (LoadBalancer :80)
                                |--> /health /predict (FastAPI)
                                |--> /metrics --scrape--> Prometheus --> Grafana
```

## 12. How to Verify

- API: curl localhost/predict -H 'Content-Type: application/json' -d @data/sample_input.json
- MLflow UI: mlflow ui -> http://localhost:5000
- Tests: pytest tests/ -v (16 tests)
- Metrics: curl localhost/metrics; Grafana at localhost:3000 (admin/admin)
- CI: github.com/2025af05103-source/mlops-assignment-1/actions - green runs